

# Rajalakshmi Engineering College

Name: M.S Nanthana shree  
Email: 241901064@rajalakshmi.edu.in  
Roll no: 241901064  
Phone: 9360753427  
Branch: REC  
Department: CSE (CS) - Section 1  
Batch: 2028  
Degree: B.E - CSE (CS)

Scan to verify results



## 2024\_28\_III\_OOPS Using Java Lab

### REC\_2028\_OOPS using Java\_Week 3\_CY

Attempt : 1  
Total Mark : 40  
Marks Obtained : 40

#### Section 1 : Coding

##### 1. Problem Statement

Rina is managing the inventory for a library, where each row of a 2D matrix represents the number of different genres of books available on each shelf.

She wants to perform the following operations:

Transformation: Replace each element in a row with the sum of all elements in that row. Merging: After transformation, Rina will provide one additional matrix, and specify whether to merge the transformed matrix with this new matrix row-wise or column-wise.

##### ***Input Format***

The first line contains two integers R and C, representing the number of rows and columns of the initial matrix.

The next R lines contain C space-separated integers, representing the book counts in the library.

The next line contains two integers MR and MC, representing the dimensions of the second matrix (to be merged).

The next MR lines contain MC space-separated integers, representing the second matrix.

The last line contains an integer mergeType:

- 0 Row-wise merging (append the second matrix below the transformed matrix).
- 1 Column-wise merging (append the second matrix to the right of the transformed matrix).

### **Output Format**

The output prints "Transformed matrix: " followed by the transformed 2D matrix where each element in a row is replaced with the sum of the elements in that row.

The output prints "Final merged matrix: ", followed by the merging based on mergeType.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 3 4

8 2 4 9

4 5 6 1

7 8 9 3

2 4

3 5 7 2

6 1 4 9

0

Output: Transformed matrix:

23 23 23 23

16 16 16 16

27 27 27 27  
Final merged matrix:  
23 23 23 23  
16 16 16 16  
27 27 27 27  
3 5 7 2  
6 1 4 9

### **Answer**

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int R = sc.nextInt();
        int C = sc.nextInt();

        int[][] matrix = new int[R][C];

        for (int i = 0; i < R; i++) {
            for (int j = 0; j < C; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        int[][] transformed = new int[R][C];
        for (int i = 0; i < R; i++) {
            int rowSum = 0;
            for (int j = 0; j < C; j++) {
                rowSum += matrix[i][j];
            }
            Arrays.fill(transformed[i], rowSum);
        }

        int MR = sc.nextInt();
        int MC = sc.nextInt();
        int[][] second = new int[MR][MC];
```

```
for (int i = 0; i < MR; i++) {  
    for (int j = 0; j < MC; j++) {  
        second[i][j] = sc.nextInt();  
    }  
}
```

```
int mergeType = sc.nextInt();
```

```
System.out.print("Transformed matrix: ");  
for (int i = 0; i < R; i++) {  
    for (int j = 0; j < C; j++) {  
        System.out.print(transformed[i][j] + " ");  
    }  
}  
System.out.println();
```

```
System.out.print("Final merged matrix: ");
```

```
if (mergeType == 0) {  
    for (int i = 0; i < R; i++) {  
        for (int j = 0; j < C; j++) {  
            System.out.print(transformed[i][j] + " ");  
        }  
    }
```

```
    for (int i = 0; i < MR; i++) {  
        for (int j = 0; j < MC; j++) {  
            System.out.print(second[i][j] + " ");  
        }  
    }
```

```
    } else {  
        int rows = Math.max(R, MR);  
        for (int i = 0; i < rows; i++) {  
            if (i < R) {  
                for (int j = 0; j < C; j++) {
```

```
        System.out.print(transformed[i][j] + " ");  
    }  
}  
  
if (i < MR) {  
    for (int j = 0; j < MC; j++) {  
        System.out.print(second[i][j] + " ");  
    }  
}  
}  
}
```

## 2. Problem Statement

Refer to the sample output for the formatting specifications.

**Sample Test Case**

Input: 2 2

1 2

3 4

Output: 1 2

**Answer**

```
import java.util.*;
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
  
        int R = sc.nextInt();  
        int C = sc.nextInt();
```

```
  
        int[][] matrix = new int[R][C];
```

```
  
        for (int i = 0; i < R; i++) {  
            for (int j = 0; j < C; j++) {  
                matrix[i][j] = sc.nextInt();  
            }  
        }
```

```
  
        int maxSum = Integer.MIN_VALUE;  
        int rowToRemove = -1;
```

```
  
        for (int i = 0; i < R; i++) {  
            int rowSum = 0;  
            for (int j = 0; j < C; j++) {  
                rowSum += matrix[i][j];  
            }  
            if (rowSum > maxSum) {  
                maxSum = rowSum;  
                rowToRemove = i;  
            }
```

```

    }
}

for (int i = 0; i < R; i++) {
    if (i == rowToRemove) continue;
    for (int j = 0; j < C; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
}
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement:

Emma, a budding computer vision enthusiast, is working on a challenging image processing project. She has a square image represented as a 2D matrix of integers. As part of a special filter operation, she needs to rotate the image by 90 degrees clockwise, but there's a twist — she must perform the rotation in-place, using no extra space.

This means Emma has to rotate the matrix without creating a new one. Your task is to help her implement a Java program that takes this square matrix as input and rotates it within the same structure.

Can you help Emma efficiently rotate the image so that her project can move to the next stage?

#### **Input Format**

The first line of input contains a single integer  $n$ , representing the number of rows and columns of the square matrix (i.e., the matrix is of size  $n \times n$ ).

The next  $n$  lines each contain  $n$  space-separated integers, representing the elements of each row of the 2D array.

#### **Output Format**

The first line of output prints "Rotated 2D Array:"

The next n lines of output print the rotated matrix.

Each line contains n space-separated integers representing a row of the rotated matrix.

Refer to the sample output for format specification.

### **Sample Test Case**

Input: 3

1 2 3

4 5 6

7 8 9

Output: Rotated 2D Array:

7 4 1

8 5 2

9 6 3

### **Answer**

```
import java.util.*;
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();  
        int[][] mat = new int[n][n];
```

```
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n; j++) {  
                mat[i][j] = sc.nextInt();  
            }  
        }
```

```
        for (int i = 0; i < n; i++) {  
            for (int j = i + 1; j < n; j++) {
```



```

        int temp = mat[i][j];
        mat[i][j] = mat[j][i];
        mat[j][i] = temp;
    }
}

```

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n / 2; j++) {
        int temp = mat[i][j];
        mat[i][j] = mat[i][n - 1 - j];
        mat[i][n - 1 - j] = temp;
    }
}

```

```

System.out.println("Rotated 2D Array:");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.print(mat[i][j] + " ");
    }
    System.out.println();
}
}
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement:

Imagine you have an array of integer values, and you're tasked with identifying a pair of elements within the array. This pair of elements should have a sum that is the closest to zero when compared to any other pair in the array.

Your goal is to create a program that solves this problem efficiently. The program should accept an array of integers and return the pair of elements whose sum is closest to zero.

**Input Format**

The first line of the input is an integer N representing the size of the array.

The second line of the input contains N space-separated integer values.

### **Output Format**

The output is displayed in the following format:

"Pair with the sum closest to zero: {value} and {value}"

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 5

9 10 -3 -5 -2

Output: Pair with the sum closest to zero: 9 and -5

### **Answer**

```
import java.util.*;
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();  
        int[] arr = new int[n];
```

```
        for (int i = 0; i < n; i++) {  
            arr[i] = sc.nextInt();  
        }
```

```
        int minSum = Integer.MAX_VALUE;  
        int first = arr[0], second = arr[1];
```

```
        for (int i = 0; i < n - 1; i++) {  
            for (int j = i + 1; j < n; j++) {  
                int sum = arr[i] + arr[j];  
                if (Math.abs(sum) < Math.abs(minSum)) {
```

```
        minSum = sum;  
        first = arr[i];  
        second = arr[j];  
    }  
}
```

```
    System.out.println("Pair with the sum closest to zero: " + first + " and " +  
second);  
}  
}
```

**Status :** Correct

**Marks : 10/10**